

Kapitel 5: Sortieren und Auswahl

SelectionSort · InsertionSort · MergeSort · QuickSort | Master-Theorem | QuickSelect

1 Grundbegriffe: Ordnungen

Definition 5.1 — Lineare Preordnung und lineare Ordnung

Eine Relation $\leq$ auf einer Schlüsselmenge K ist eine **lineare Preordnung**, falls:

- **Totalität:** $\forall a, b \in K : a \leq b \vee b \leq a$
- **Transitivität:** $a \leq b \wedge b \leq c \Rightarrow a \leq c$

Eine **lineare Ordnung** ist zusätzlich **antisymmetrisch**: $a \leq b \wedge b \leq a \Rightarrow a = b$.

Lexikografische Ordnung auf K^n : $(a_1, \dots, a_n) \leq (b_1, \dots, b_n)$, falls ein i existiert mit $a_i < b_i$ und $a_j = b_j$ für alle $j < i$. Anwendung: Wörterbücher, Telefonbücher.

Sortierproblem: Gegeben Array $a[0..n-1]$ mit Schlüsseln aus K — bringe es in eine Reihenfolge, sodass $i \leq j \Rightarrow a[i].\text{key} \leq a[j].\text{key}$.

Für verkettete Listen genügt Transitivität (kein wahlfreier Zugriff nötig).

Definition 5.2 — Stabilität und In-place

Stabil: Elemente mit *gleichem* Schlüssel behalten ihre relative Reihenfolge aus der Eingabe.

In-place: Der Algorithmus benötigt nur $\mathcal{O}(1)$ Zusatzspeicher (neben der Eingabe selbst).

Algorithmus	Stabil	In-place	Best Case	Worst / Avg Case
SelectionSort	✗	✓	$\Theta(n^2)$	$\Theta(n^2)$
InsertionSort	✓	✓	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$ / $\Theta(n + D)$
MergeSort	✓	✗	$\Theta(n \log n)$	$\Theta(n \log n)$
QuickSort	✗	✓	$\Theta(n \log n)$	$\mathcal{O}(n^2)$ WC / $\mathcal{O}(n \log n)$ $\emptyset$

2 SelectionSort

Algorithmus 5.3 — SelectionSort

Idee: Wiederhole n mal: finde das Minimum im unsortierten Restteil und hänge es ans sortierte Ende an.

Invariante: Nach Runde k sind die k kleinsten Elemente korrekt am Anfang sortiert.

Komplexität:

- Minimum suchen: $\mathcal{O}(n - k)$ in Runde $k \Rightarrow$ Gesamt: $\sum_{k=0}^{n-1} (n - k) = \frac{n(n+1)}{2} \in \Theta(n^2)$
- **Immer** $\Theta(n^2)$ — die Eingabe spielt keine Rolle
- In-place mit Arrays, aber **nicht stabil** (Swap kann Reihenfolge gleicher Elemente zerstören)

Verbesserung: Minimum-Suche mit Heap $\Rightarrow$ **HeapSort** in $\mathcal{O}(n \log n)$ (in-place, nicht stabil).

Beispiel 5.4 — SelectionSort auf [5, 3, 1, 4, 2]

Runde	Unsortiert	Minimum	Ergebnis
1	[5, 3, 1, 4, 2]	1	[1 5, 3, 4, 2]
2	[5, 3, 4, 2]	2	[1, 2 5, 3, 4]
3	[5, 3, 4]	3	[1, 2, 3 5, 4]
4	[5, 4]	4	[1, 2, 3, 4 5]
5	[5]	5	[1, 2, 3, 4, 5]

3 InsertionSort

Algorithmus 5.5 — InsertionSort und Inversionen

Idee: Füge das k -te Element an die korrekte Position im bereits sortierten Präfix $a[0..k-1]$ ein (wie Kartenhand sortieren — von links nach rechts verschieben).

Inversion: Ein Paar (i, j) mit $i < j$ aber $a[i] > a[j]$.

Satz: InsertionSort benötigt genau $\Theta(n + D)$ Schritte, wobei $D = \sum_i |\text{rang}(i) - i|$ die **Anzahl der Inversionen** ist.

Fall	Inversionen D	Komplexität
Best Case (bereits sortiert)	$D = 0$	$\mathcal{O}(n)$
Average Case (zufällig)	$D = \Theta(n^2)$	$\mathcal{O}(n^2)$
Worst Case (umgekehrt sortiert)	$D = \frac{n(n-1)}{2}$	$\Theta(n^2)$
Fast sortiert	D klein	$\mathcal{O}(n + D)$

Binäre Suche: Einfügeposition in $\mathcal{O}(\log n)$ findbar — aber das Verschieben der Elemente bleibt $\mathcal{O}(n)$, also insgesamt keine Verbesserung für Arrays. Funktioniert *nicht* für verkettete Listen (kein wahlfreier Zugriff).

Beispiel 5.6 — InsertionSort auf [4, 2, 5, 1, 3]

Schritt	Aktion	Array
$k = 1$	2 einfügen vor 4	[2, 4, 5, 1, 3]
$k = 2$	5 bleibt	[2, 4, 5, 1, 3]
$k = 3$	1 ganz nach vorne	[1, 2, 4, 5, 3]
$k = 4$	3 zwischen 2 und 4	[1, 2, 3, 4, 5]

Inversionen: (4, 2), (4, 1), (4, 3), (2, 1), (5, 1), (5, 3) $\Rightarrow D = 6$.

4 MergeSort

Algorithmus 5.7 — MergeSort (Divide and Conquer)

Prinzip:

1. **Split:** Teile die Liste in zwei gleich große Hälften — $\mathcal{O}(1)$ mit `splite` (Liste) bzw. $\mathcal{O}(1)$ mit Indexberechnung (Array).
2. **Rekursion:** Sortiere beide Hälften rekursiv.
3. **Merge:** Führe zwei sortierte Folgen zusammen — immer das kleinere Frontelement wählen, $\mathcal{O}(n_1 + n_2)$.

Rekurrenz und Analyse:

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n) \xrightarrow{\text{Master-Thm, Fall 2}} T(n) = \Theta(n \log n)$$

Eigenschaften:

- ✓ Stabil ✓ $\Theta(n \log n)$ für *jede* Eingabe (kein schlechter Fall)
- ✗ Nicht in-place für Arrays (braucht $\mathcal{O}(n)$ Zusatzspeicher für Merge)
- ✓ In-place für verkettete Listen (nur Zeiger umhängen via `splite`)
- Python `list.sort()` und `sorted()` nutzen **Timsort** (hochoptimiertes MergeSort)

Beispiel 5.8 — Merge zweier sortierten Folgen

Merge $A = [1, 4, 6]$ und $B = [2, 3, 5]$:

$$\underbrace{1}_{\text{aus } A} \rightarrow \underbrace{2}_{\text{aus } B} \rightarrow \underbrace{3}_{\text{aus } B} \rightarrow \underbrace{4}_{\text{aus } A} \rightarrow \underbrace{5}_{\text{aus } B} \rightarrow \underbrace{6}_{\text{aus } A} \Rightarrow [1, 2, 3, 4, 5, 6]$$

Jedes Element wird genau einmal verglichen und entnommen: $\mathcal{O}(n_1 + n_2) = \mathcal{O}(6)$ Schritte.

5 Master-Theorem

Satz 5.9 — Master-Theorem

Gegeben die Rekurrenz $T(n) = aT(n/b) + f(n)$ mit $a \geq 1$, $b > 1$. Sei $c = \log_b a$.

1. **Fall 1** — Rekursionsbaum dominiert: $f(n) \in \mathcal{O}(n^{c-\varepsilon})$ für ein $\varepsilon > 0$

$$\Rightarrow T(n) = \Theta(n^c)$$

2. **Fall 2** — gleich stark: $f(n) \in \Theta(n^c)$

$$\Rightarrow T(n) = \Theta(n^c \log n)$$

3. **Fall 3** — Wurzel dominiert: $f(n) \in \Omega(n^{c+\varepsilon})$ und $a f(n/b) \leq \delta f(n)$ für $\delta < 1$

$$\Rightarrow T(n) = \Theta(f(n))$$

Merkhilfe: Vergleiche $f(n)$ mit $n^{\log_b a}$ — wer wächst schneller?
 f kleiner $\rightarrow$ Fall 1; f gleich $\rightarrow$ Fall 2 ($+\log n$); f größer $\rightarrow$ Fall 3.

Beispiel 5.10 — Master-Theorem anwenden

Rekurrenz	a	b	$c = \log_b a$	Ergebnis
MergeSort: $2T(n/2) + n$	2	2	1	$f = n = n^1 \rightarrow$ Fall 2 $\rightarrow \Theta(n \log n)$
$T(n) = 3T(n/4) + n^2$	3	4	$\approx 0,79$	$f = n^2 \gg n^{0.79} \rightarrow$ Fall 3 $\rightarrow \Theta(n^2)$
$T(n) = T(n/2) + 1$	1	2	0	$f = 1 = n^0 \rightarrow$ Fall 2 $\rightarrow \Theta(\log n)$
$T(n) = 4T(n/2) + n$	4	2	2	$f = n \ll n^2 \rightarrow$ Fall 1 $\rightarrow \Theta(n^2)$

6 QuickSort

Algorithmus 5.11 — QuickSort

Prinzip: Divide and Conquer — Arbeit beim *Teilen*, Zusammenführen trivial.

1. **Pivot** p wählen (z. B. letztes Element oder zufällig).
2. **Partition:** Trenne in $L = \{x \mid x \leq p\}$ und $R = \{x \mid x \geq p\}$ — in $\mathcal{O}(n)$.
3. Rekursiv L und R sortieren; Ergebnis: $[L, p, R]$ (in-place).

Komplexitätsanalyse:

Fall	Pivot-Wahl	Partition	Komplexität
Worst Case	stets Max oder Min	$ L = n-1, R = 0$	$\Theta(n^2)$
Best Case	stets Median	$ L = R = n/2$	$\Theta(n \log n)$
Average / randomisiert	zufällig	erwartet ausgewogen	$\mathcal{O}(n \log n)$

Worst-Case-Herleitung:

$$T(n) = T(n-1) + \mathcal{O}(n) = \mathcal{O}(n) + \mathcal{O}(n-1) + \cdots + \mathcal{O}(1) = \mathcal{O}(n^2)$$

Durchschnittliche Komplexität (Argument über Vergleiche):

- Ein Pivot ist *gut*, wenn er im mittleren 50%-Bereich liegt (Partitionen $\geq n/4$).
- $\Pr[\text{Pivot gut}] = \frac{1}{2}$ bei zufälliger Wahl.
- Erwartete Tiefe des Rekursionsbaums: $\mathcal{O}(\log n)$, Arbeit pro Ebene: $\mathcal{O}(n)$.
- $\Rightarrow \mathbb{E}[\text{Vergleiche}] = \mathcal{O}(n \log n)$.

Randomisierung: Pivot zufällig wählen $\Rightarrow$ Worst Case mit hoher Wahrscheinlichkeit vermieden, *für jede* Eingabe.

Beispiel 5.12 — QuickSort-Partition auf $[3, 6, 8, 10, 1, 2, 1]$, Pivot = 1

Vor Partition: $[3, 6, 8, 10, 1, 2, 1]$, Pivot $p = 1$ (letztes Element)

Partition: $L = []$, Pivot = 1, $R = [3, 6, 8, 10, 2, 1]$ (vereinfacht)

Nächster Schritt: Rekursion auf R mit günstigem Pivot $\rightarrow \mathcal{O}(n \log n)$ erwartet.

Worst-Case-Beispiel: Array $[1, 2, 3, 4, 5]$, Pivot stets letztes Element (Maximum):

Partition ergibt $L = [1, 2, 3, 4]$, $R = [] \Rightarrow$ Tiefe $n \Rightarrow \Theta(n^2)$.

Achtung

QuickSort vs. MergeSort im Vergleich:

	QuickSort	MergeSort
Worst Case	$\Theta(n^2)$	$\Theta(n \log n)$
Average	$\mathcal{O}(n \log n)$	$\Theta(n \log n)$
In-place	✓	✗ (Arrays)
Stabil	✗	✓
Cache	gut (sequenziell)	mittel
Praxis	oft schneller	stabiler

Python `list.sort()` = **Timsort** (MergeSort-basiert, stabil). C++ `std::sort` = **Introsort** (Quick + Heap + Insertion).

7 QuickSelect — Auswahlproblem

Algorithmus 5.13 — QuickSelect

Problem: Finde das k -te kleinste Element (z. B. den Median) in einer unsortierten Folge.

Idee: Wie QuickSort, aber *einseitige* Rekursion — nach der Partition wird nur die Seite weiterverfolgt, die den Rang k enthält.

1. Partition mit Pivot p : ergibt $|L|$ und $|R|$.
2. Falls $k \leq |L|$: suche k -tes Element rekursiv in L .
3. Falls $k = |L| + 1$: p ist das k -te Element — fertig.
4. Sonst: suche $(k - |L| - 1)$ -tes Element rekursiv in R .

Komplexität:

Fall	Rekurrenz	Komplexität
Worst Case (schlechter Pivot)	$T(n) = T(n-1) + \mathcal{O}(n)$	$\mathcal{O}(n^2)$
Best Case (Median als Pivot)	$T(n) = T(n/2) + \mathcal{O}(n)$	$\mathcal{O}(n)$ (Master Fall 3)
Average (zufälliger Pivot)	—	$\mathcal{O}(n)$ (erwartet)

⇒ Mit randomisiertem Pivot: **erwartet** $\mathcal{O}(n)$ — deutlich besser als erst sortieren ($\mathcal{O}(n \log n)$).

Beispiel 5.14 — QuickSelect: Median von $[3, 1, 4, 1, 5, 9, 2, 6]$

$n = 8$, gesucht: $k = 4$ (4. kleinstes Element = Median).

Wähle Pivot $p = 6$. Partition: $L = [3, 1, 4, 1, 5, 2]$, $R = [9]$.

$|L| = 6 \geq k = 4 \Rightarrow$ rekursiv in L nach dem 4. kleinsten Element suchen.

Wähle $p = 4$: $L' = [3, 1, 1, 2]$, $R' = [5]$. $|L'| = 4 = k \Rightarrow p = 4$ ist der Median. ✓

Prüfungsscheckliste — Kapitel 5: Sortieren und Auswahl

- ☐ Lineare Preordnung vs. lineare Ordnung: Eigenschaften (Totalität, Transitivität, Antisymmetrie) nennen. → *Def. 5.1*
- ☐ Lexikografische Ordnung definieren und Anwendung nennen. → *Def. 5.1*
- ☐ Stabilität und In-place definieren; Vergleichstabelle der 4 Algorithmen auswendig. → *Def. 5.2, §1*
- ☐ SelectionSort: Ablauf + Invariante, Komplexität $\Theta(n^2)$ herleiten (Summenformel). → *Alg. 5.3*
- ☐ Warum ist SelectionSort nicht stabil? Gegenbeispiel angeben. → *Alg. 5.3*
- ☐ Verbindung SelectionSort → HeapSort: was ändert sich? → *Alg. 5.3*
- ☐ Inversion definieren; Satz $T = \Theta(n + D)$ für InsertionSort formulieren. → *Alg. 5.5*
- ☐ InsertionSort Best/Average/Worst Case mit konkretem D -Wert angeben. → *Alg. 5.5*
- ☐ Warum hilft binäre Suche bei InsertionSort für Arrays nicht? Und für Listen? → *Alg. 5.5*
- ☐ MergeSort: alle drei Schritte (Split, Rekursion, Merge) erklären. → *Alg. 5.7*
- ☐ Merge zweier sortierter Folgen: Algorithmus und Komplexität $\mathcal{O}(n_1 + n_2)$ begründen. → *Alg. 5.7, Bsp. 5.8*
- ☐ MergeSort-Rekurrenz aufstellen und mit Master-Theorem lösen (Fall 2). → *Alg. 5.7*
- ☐ Warum ist MergeSort für Listen in-place, für Arrays aber nicht? → *Alg. 5.7*
- ☐ Master-Theorem: Formel, drei Fälle, $c = \log_b a$ berechnen. → *Satz 5.9*
- ☐ Master-Theorem auf 4 Beispielrekurrenzen anwenden — inklusive Begründung des Falls. → *Bsp. 5.10*

-
- QuickSort: Partition erklären, Worst/Best/Average-Case herleiten. → *Alg. 5.11*
 - Worst-Case von QuickSort herleiten: $T(n) = T(n-1) + \mathcal{O}(n) \Rightarrow \mathcal{O}(n^2)$. → *Alg. 5.11*
 - Warum hilft Randomisierung? Wahrscheinlichkeit guter Pivot erklären. → *Alg. 5.11*
 - QuickSort vs. MergeSort: alle Unterschiede (Worst, Stabil, In-place, Cache) benennen. → *Achtung, §6*
 - QuickSelect: Idee, einseitige Rekursion, Komplexitäten (WC/Best/Avg). → *Alg. 5.13*
 - Warum ist QuickSelect mit $\mathcal{O}(n)$ erwartet besser als erst sortieren ($\mathcal{O}(n \log n)$)? → *Alg. 5.13*
-

Literatur: DMS §5 · CLRS §2, §4, §7.1, §9.2